

Activité 1

Considérez la classe Dart suivante :

```
class Personne {  
    String name;  
    String _prenom;  
    int age;  
    Personne(this.name, this._prenom, this.age);  
    void afficher_nom() {  
        print("Je m'appelle: $name, mon prénom est $_prenom, j'ai $age ans");  
    }  
}
```

- Ajoutez les méthodes **getter** et **setter** pour les variables d'instance **name**, **_prenom** et **age** de la classe **Personne**.
- Les méthodes **getter** doivent retourner la valeur respective de chaque variable et les méthodes **setter** doivent modifier la valeur de chaque variable.
- Vérifiez que les méthodes **getter** et **setter** fonctionnent correctement en effectuant les étapes suivantes :
 - Créez une instance de la classe **Personne**.
 - Utilisez les méthodes **setter** pour modifier les valeurs des variables d'instance.
 - Utilisez les méthodes **getter** pour accéder aux valeurs des variables d'instance.
 - Appelez la méthode **afficher_nom()** pour afficher le **nom**, le **prénom** et l'**age**.
- Vous devez compléter la classe **Personne** en ajoutant les méthodes **getter** et **setter** appropriées pour chaque variable d'instance. Assurez-vous de comprendre comment fonctionnent les méthodes **getter** et **setter** en Dart.

Activité 2

- Créez une classe appelée **Point** permettant de manipuler un point d'un plan (**x,y**). Vous fournirez :
 - Un constructeur recevant en argument les coordonnées (double) d'un point.

- Une méthode membre « move » effectuant une translation définie par ses deux arguments (double).
 - Une méthode membre « display » affichant simplement les coordonnées du point.
- Les coordonnées du point seront des données privées membres.
- Nous écrirons séparément :
 - Un fichier source constituant la déclaration de la classe.
 - Un petit programme de test (principal) déclarant un point, l'affichant, le déplaçant et l'affichant à nouveau.

Activité 3

- On a pour objectif de développer un **programme en Dart** pour gérer les coûts des enseignants d'une université.
- Chaque enseignant est caractérisé par son nom, prénom et le nombre d'heures de cours qu'il assure dans l'année. Il existe plusieurs catégories d'enseignants :
 - Les enseignants-chercheurs, qui reçoivent un salaire fixe mensuel ainsi qu'une rémunération pour les heures complémentaires.
 - Les vacataires, recrutés par des organismes externes à l'université, qui sont payés un tarif horaire fixe.
 - Les doctorants, qui peuvent donner des cours jusqu'à un certain nombre d'heures et sont rémunérés selon un tarif horaire.
 - Les enseignants-chercheurs reçoivent un salaire mensuel fixe et un montant supplémentaire pour les heures complémentaires au-delà d'un certain seuil. Les vacataires perçoivent un tarif horaire fixe pour toutes leurs heures. Les doctorants sont payés un tarif horaire fixe, mais leur nombre d'heures de cours est limité.
 - Pour chaque enseignant, le programme doit calculer le coût total pour l'université, en tenant compte du salaire et des charges applicables sur ce salaire.
 - Nous allons élaborer une solution en Dart qui respecte les principes de la programmation orientée objet pour modéliser ces différentes catégories d'enseignants et calculer leurs coûts respectifs.

Activité 4

- Dans cet exercice, vous devez mettre en pratique vos connaissances sur les méthodes **getter** et **setter** en Dart en réalisant un ensemble de classes pour un **système de gestion d'une bibliothèque**.
- **Instructions :**
 - Définissez une classe Livre avec les propriétés suivantes :

- **titre** : une chaîne de caractères représentant le titre du livre.
- **auteur** : une chaîne de caractères représentant le nom de l'auteur du livre.
- **anneePublication** : un entier représentant l'année de publication du livre.
- Ajoutez les méthodes getter et setter pour chaque propriété de la classe Livre. Les méthodes getter doivent retourner la valeur respective de chaque propriété et les méthodes setter doivent modifier la valeur de chaque propriété.
- Définissez une classe Bibliotheque avec les propriétés suivantes :
 - **nom** : une chaîne de caractères représentant le nom de la bibliothèque.
 - **livres** : une liste de livres (instances de la classe Livre) représentant les livres présents dans la bibliothèque.
- Ajoutez les méthodes getter et setter pour la propriété nom de la classe Bibliotheque. Les méthodes getter doivent retourner la valeur de la propriété et les méthodes setter doivent modifier la valeur de la propriété.
- Ajoutez une méthode ajouterLivre à la classe Bibliotheque qui prend en paramètre un livre et l'ajoute à la liste des livres de la bibliothèque.
- Ajoutez une méthode afficherLivres à la classe Bibliotheque qui affiche les informations de tous les livres présents dans la bibliothèque (titre, auteur, année de publication).
- Votre objectif est de créer les classes Livre et Bibliotheque, d'implémenter les méthodes getter et setter appropriées, d'ajouter des livres à la bibliothèque et d'afficher les informations des livres.

Activité 5

- Vous devez implémenter un système de gestion de compte bancaire en Dart en utilisant des méthodes **getter** et **setter** avancées.
- Définissez une classe **CompteBancaire** avec les propriétés suivantes :
 - **solde** : un double représentant le solde du compte.
 - **plafond** : un double représentant le plafond de découvert autorisé pour le compte.

- Ajoutez une méthode **getter** pour la propriété **solde**. Cette méthode doit afficher le solde actuel du compte et indiquer si le compte est à découvert.
- Ajoutez une méthode **setter** pour la propriété **solde**. Cette méthode doit vérifier si le nouveau solde dépasse le plafond de découvert autorisé. Si c'est le cas, elle doit refuser la transaction et afficher un message approprié.
- Ajoutez une méthode dépôt à la classe CompteBancaire qui prend en paramètre un montant et l'ajoute au solde du compte.
- Ajoutez une méthode retrait à la classe CompteBancaire qui prend en paramètre un montant et le retire du solde du compte. Assurez-vous que cette méthode respecte le plafond de découvert autorisé.
- Créez une instance de la classe **CompteBancaire** et effectuez une série de dépôts et de retraits pour tester votre implémentation. Assurez-vous que les méthodes **getter** et **setter** fonctionnent correctement et que les règles du plafond de découvert sont respectées.